Implementation Plan for Smart Baby Room Monitor

This section explains how the Smart Baby Room Monitor was developed and integrated. The implementation followed an incremental approach similar to **Agile Development Methodology**, where each hardware component and software feature was added, tested, and improved step by step.

The system was first built in Wokwi using an ESP32, sensors, and actuators to simulate the baby room environment. After the local monitoring and automation functions were working, the project was connected to AWS IoT Core using Wi-Fi and MQTT. Cloud services such as DynamoDB, Lambda, API Gateway, SNS, and Amplify were then added to support data storage, dashboard access, remote control, and email alerts. This approach helped reduce errors, made testing easier, and ensured that each part of the IoT system worked before full integration.

Hardware Simulation in Wokwi Environment

The hardware prototype was built in Wokwi using an ESP32 as the main controller. The circuit included sensors for temperature, humidity, light, motion, noise simulation, and safe-zone distance. These were connected to output devices such as LEDs, buzzer, OLED display, servo motors, and a heater indicator.

A table highlighting each sensor and actuator adding to the project, along side which GPIO Pin they have been connected to in the ESP32 and the reason to include it in the project, is shared in **Appendix J**.

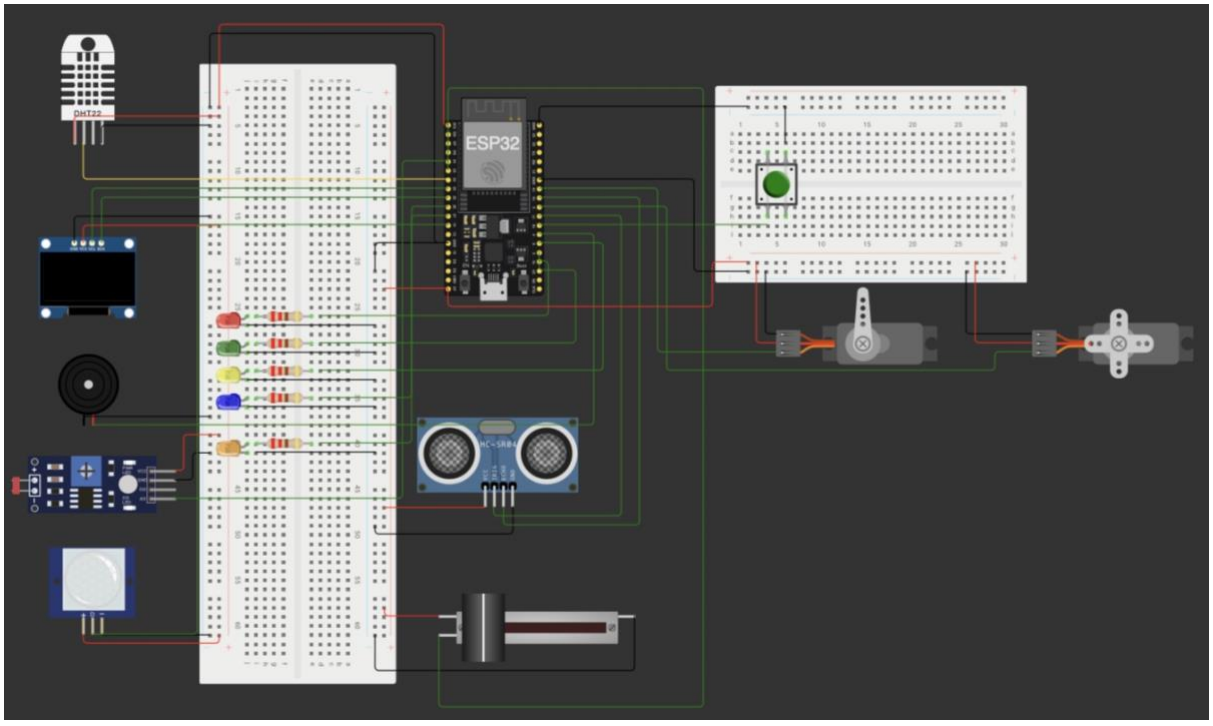


Figure 1 WOKWI circuit.

The simulation also allowed different room conditions to be created manually, such as high temperature, low light, crying/noise, movement, and unsafe distance. This made it possible to check whether the ESP32 responded correctly before connecting the system to AWS. For output visualisation, an OLED screen has been added to the hardware list.

Local Logic

The ESP32 firmware was developed using C++ within the Arduino framework. This was chosen because Wokwi supports Arduino-based ESP32 simulation, and the Arduino framework provides simple libraries for sensors, OLED display, servo motors, Wi-Fi, and MQTT communication. Below is a list of all libraries used in this project.

- DHT sensor library
- Adafruit Unified Sensor
- Adafruit SSD1306
- Adafruit GFX Library
- ESP32Servo
- PubSubClient
- ArduinoJson



Figure 2 OLED displaying room condition. UNSAFE

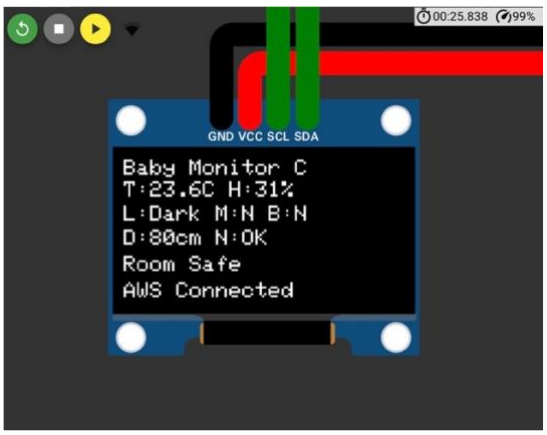


Figure 3 OLED displaying room condition. SAFE

Cloud Integration

The cloud part of the system was implemented using AWS to allow the ESP32 device, database, dashboard, and notification service to communicate with each other. AWS IoT Core was the first service configured because it acts as the MQTT broker for the project. A Thing was created to represent the ESP32 device, and certificates and an IoT policy were attached so the device could connect securely to AWS.

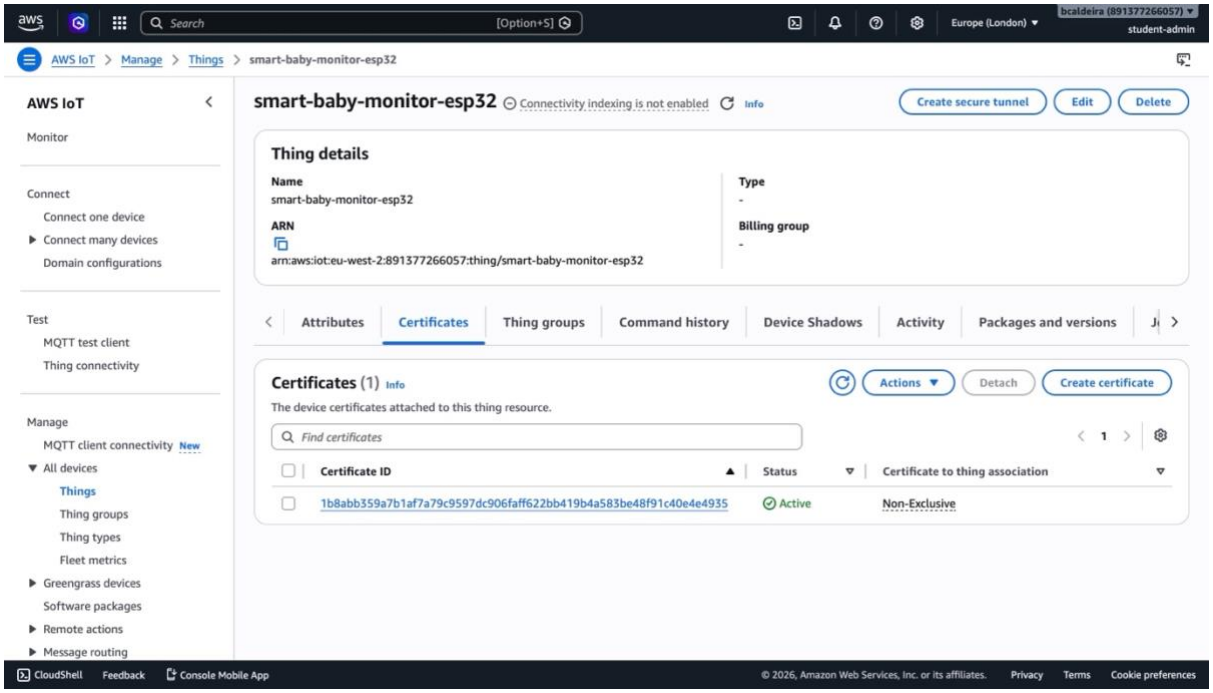


Figure 4 IoT Core in AWS console displaying created thing.

The ESP32 publishes sensor readings to the MQTT topic `baby-monitor/readings`. These readings include values such as temperature, humidity, light level, motion, noise, safe-zone distance, fan status, heater status, crib status, and alert condition. An IoT Rule was created to listen to this topic and forward incoming messages to DynamoDB for storage.

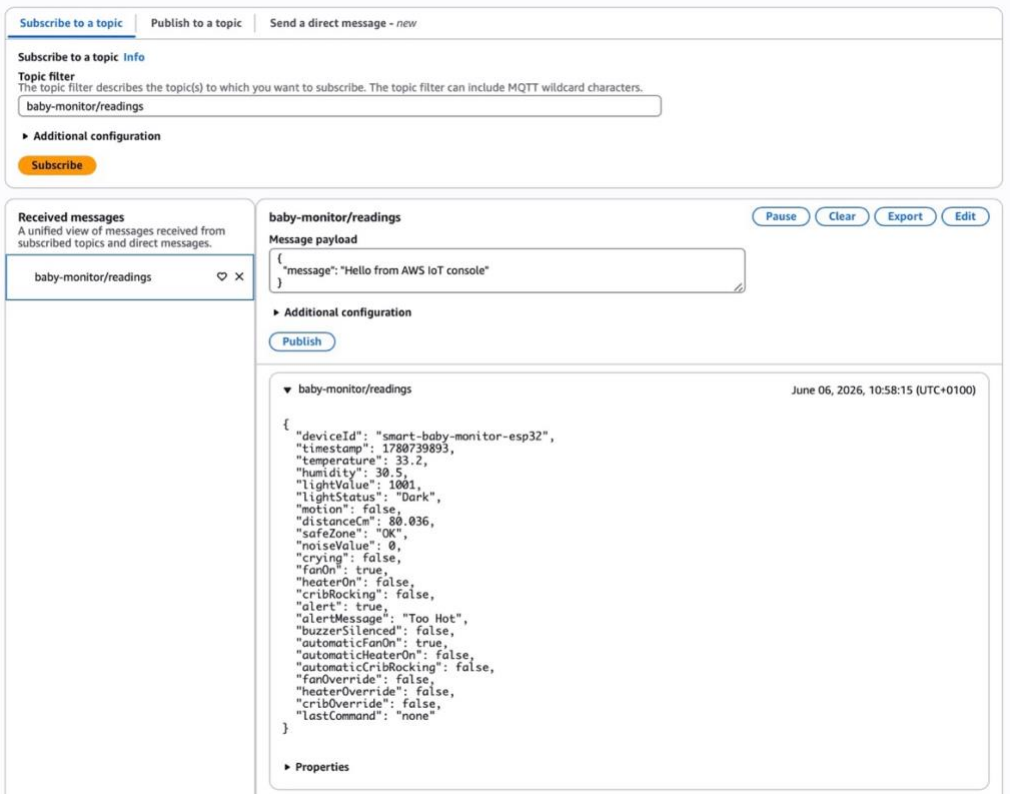


Figure 5 MQTT Test Client subscribed to baby-monitor/readings topic in AWS console.

DynamoDB was used as the project database because it is serverless, scalable, and suitable for timestamped IoT data. The table `BabyMonitorReadings` stores each reading using the `deviceId` as the partition key and `timestamp` as the sort key. This makes it possible to retrieve the latest reading and recent historical data for the dashboard.



Figure 6 Explore item section in DynamoDB in BabyMonitorReadings table.

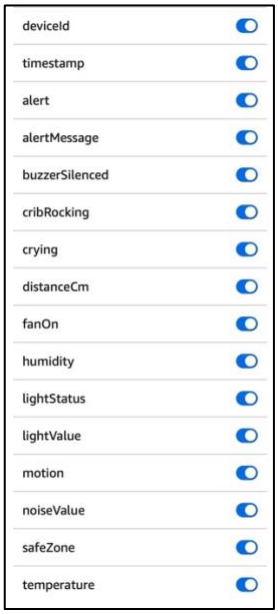


Figure 7 Values stored in BabyMonitorReadings table.

AWS Lambda functions were used to process backend logic. Their respective codes are shared in **Appendix E**.

- GetBabyMonitorReadings retrieves readings from DynamoDB.
- SendBabyMonitorCommand publishes remote commands back to AWS IoT Core.
- ProcessBabyMonitorAlert sends alert messages through SNS.
- SubscribeBabyMonitorAlerts allows users to subscribe their email address.

API Gateway was used to expose HTTP endpoints for the web and mobile dashboards. Amazon SNS was used to send email alerts when unsafe conditions were detected. Finally, AWS Amplify was used to host the React web dashboard, allowing users to access the system through a public web link, generating the below URL to host the website dashboard:

https://production.dvvxbfugcqfz7.amplifyapp.com

User Dashboard

The user dashboard was implemented in two formats: a React web dashboard and a mobile dashboard built with Expo and React Native. Both dashboards use the same AWS backend, meaning they access the same data and follow the same communication flow. This avoids duplication and ensures that users receive consistent information whether they access the system from a browser or a smartphone.

The dashboards communicate with API Gateway, which connects to Lambda functions. Sensor readings are retrieved through the `GetBabyMonitorReadings` function, which reads the latest and recent data from DynamoDB. Remote commands, such as reset, fan control, heater control, and crib rocking, are sent through the `SendBabyMonitorCommand` function and then published to AWS IoT Core for the ESP32 to receive.

Both dashboards display the current room status, temperature, humidity, noise level, safe-zone condition, night light status, crib status, fan status, and heater status. The web dashboard also includes a temperature trend using the last ten readings, while the mobile dashboard provides a more convenient option for quick monitoring on a smartphone.

This implementation improves usability because parents can monitor the baby room and respond remotely using the device most convenient to them. It also makes the system more maintainable, as both dashboard options rely on the same APIs, cloud services, and data flow.

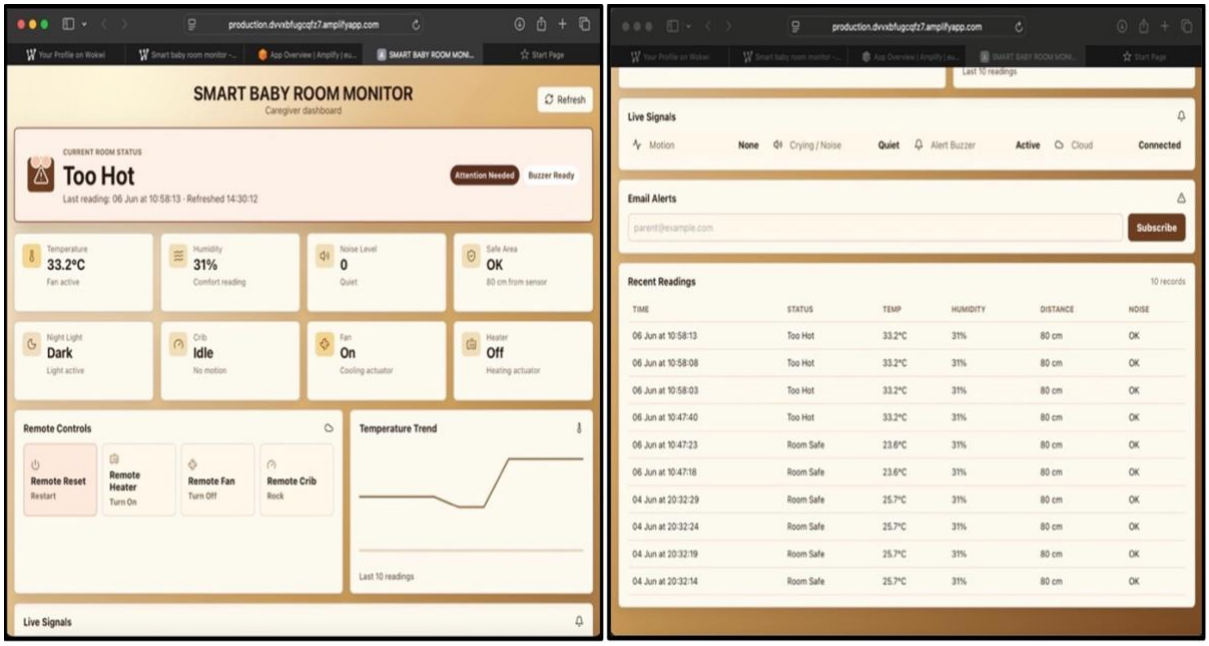


Figure 8 WebSite User Interface



Figure 9 Mobile App User Interface

Security and Monitoring

The main security and privacy risks in this project are unauthorised access, data interception, device impersonation, data leakage, and misuse of remote commands. To reduce these risks, the ESP32 connects to AWS IoT Core using TLS encryption and X.509 certificates, helping protect MQTT communication and ensuring that only the registered device can connect. An IoT policy controls which topics the ESP32 can publish and subscribe to, while IAM roles limit what each Lambda function can access. API Gateway is used as a controlled entry point for the web and mobile dashboards, preventing direct access to DynamoDB or AWS IoT Core. The system also avoids using a camera or recording audio, reducing privacy risks because only sensor readings and alert states are stored.

Monitoring is also important for maintaining security. AWS services such as CloudWatch can be used to monitor Lambda activity, errors, API requests, and system behaviour. This allows unusual activity, failed requests, or repeated alert triggers to be identified more easily. In a real deployment, monitoring logs would help detect possible misuse, support troubleshooting, and provide an audit trail of important system events.

Usability and Scalability

The current version is user-friendly and the main functions are ready for user testing. Parents can view room status, check sensor readings, receive email alerts, and send remote commands through the web or mobile dashboard.

However, the current system is not fully scalable because it supports one ESP32 device and one baby room. A future version could support multiple rooms by registering each ESP32 as a separate AWS IoT Thing, using unique device IDs and MQTT topics. DynamoDB could store readings by `deviceId`, while user authentication could be added with Amazon Cognito. API Gateway and Lambda could then return only the devices linked to each authenticated user. This would make the system suitable for families, nurseries, or childcare providers with multiple monitored rooms.

Appendix J – ESP32 PINS List

Hardware	GPIO Pin	Reason For Adding
DHT22 temperature/humidity sensor	GPIO32	Measures room temperature and humidity so the system can monitor comfort and trigger fan/heater responses.
Photoresistor / LDR	GPIO34	Detects light level so the night light can respond automatically when the room is dark.
PIR motion sensor	GPIO14	Detects baby movement or movement near the crib.
HC-SR04 ultrasonic trigger	GPIO17	Sends ultrasonic pulse for safe-zone distance measurement.
HC-SR04 ultrasonic echo	GPIO18	Receives ultrasonic echo to calculate distance from the crib/safe area.
Potentiometer	GPIO35	Simulates baby crying/noise level in Wokwi.
OLED display SDA	GPIO25	Sends display data from ESP32 to OLED screen using I2C.
OLED display SCL	GPIO33	Provides I2C clock signal for the OLED display.
Green LED	GPIO2	Shows that the room condition is safe.
Red LED	GPIO15	Shows alert or unsafe room condition.
Amber LED / night light	GPIO0	Simulates automatic night light when the room is dark.

Buzzer	GPIO4	Provides local alert feedback during unsafe conditions in the simulation.
Push button	GPIO16	Allows local reset or buzzer silence function.
Crib servo motor	GPIO19	Simulates automatic crib rocking when movement is detected.
Fan servo motor	GPIO5	Simulates fan activation when temperature is high.
Wi-Fi/AWS status LED	GPIO26	Shows whether the ESP32 is connected to Wi-Fi/AWS IoT Core.
Heater LED	GPIO27	Simulates heater activation when the room temperature is too low.